Haskell. Часть 3

Выполнение и взаимодействие

Андреев Александр
Крапивина Ульяна
Савосина Дарья

Чистые функции и действия IO

01 Чистая функция вычисляет результат по аргументам.

02 IO a описывает действие, результат которого имеет тип a.

03 Среда выполнения исполняет действие main.

```
greet :: String -> String
greet name = "Hello, " ++ name

main :: IO ()
main = do
  name <- getLine
  putStrLn (greet name)
```

IO () обозначает действие с результатом (), например вывод строки.

Синтаксис do и связывание действий

01 <- связывает имя с результатом действия.

02 let задаёт обычное локальное выражение.

03 >>= передаёт результат следующему действию.

```
main = do
  name <- getLine
  let message = "Hi, " ++ name
  putStrLn message

-- Та же последовательность
main2 = getLine >>= \name ->
  putStrLn ("Hi, " ++ name)
```

В блоке IO действия связаны последовательно. Сам do используется и с другими монадами.

Монады на знакомом примере Maybe

01 Монада описывает последовательную композицию вычислений в контексте.

02 pure помещает значение в контекст.

03 >>= учитывает контекст при передаче результата.

```
half :: Int -> Maybe Int
half n
  | even n    = Just (n `div` 2)
  | otherwise = Nothing

quarter n = do
  x <- half n
  half x
-- quarter 8 = Just 2
-- quarter 6 = Nothing
```

Maybe прекращает цепочку при Nothing. IO связывает действия с внешними эффектами.

Консольный ввод и вывод

01 getLine читает строку без завершающего перевода строки.

02 putStrLn выводит строку, print использует Show.

03 stdin, stdout и stderr являются стандартными дескрипторами.

```
import System.IO
  (hPutStrLn, stderr)

main = do
  line <- getLine
  print (length line)
  hPutStrLn stderr "done"
```

Для строки abc: stdout содержит 3, stderr содержит done.

Файлы и режимы доступа

01 writeFile перезаписывает файл, appendFile дописывает.

02 readFile возвращает ленивую строку.

03 Handle позволяет управлять режимом и кодировкой.

```
import System.IO

main = do
  writeFile "demo.txt" "one\n"
  appendFile "demo.txt" "two\n"
  withFile "demo.txt" ReadMode $ \h -> do
    hSetEncoding h utf8
    line <- hGetLine h
    putStrLn line
-- one
```

Для реального текста кодировку стоит задавать явно. WriteMode очищает существующее содержимое.

Ресурсы и ленивое чтение

01 withFile закрывает Handle после завершения действия, включая исключения.

02 Отложенное чтение должно завершиться до закрытия Handle.

03 evaluate (length text) потребляет всю конечную строку внутри блока.

```
import Control.Exception (evaluate)
import System.IO

countChars :: FilePath -> IO Int
countChars path =
  withFile path ReadMode $ \h -> do
    text <- hGetContents h
    evaluate (length text)
```

Возврат hGetContents из withFile может оставить чтение на момент, когда файл уже закрыт.

Композиция чистой обработки

01 (.) объединяет функции без промежуточных имён.

02 filter выбирает элементы, map преобразует, sum сворачивает.

03 Чистое ядро удобно проверять отдельно от IO.

```
process :: [Int] -> Int
process = sum . map (* 2) . filter even

-- process [1..10] = 60

main = print (process [1..10])
```

Композиция читается справа налево. Сам оператор (.) не запускает параллельное выполнение.

Конкурентность и параллелизм

01 Конкурентность позволяет нескольким задачам продвигаться независимо.

02 Параллелизм означает одновременное выполнение вычислений.

03 На одном ядре задачи могут чередоваться.

```
-- Конкурентные действия IO
forkIO :: IO () -> IO ThreadId

-- Подсказка для чистых вычислений
par :: a -> b -> b

-- Два runtime capability
-- app +RTS -N2 -RTS
```

Потоки IO полезны для взаимодействия. Чистые независимые вычисления можно распараллеливать.

forkIO и ожидание результата

01 forkIO возвращается после запуска потока.

02 MVar передаёт результат и синхронизирует задачи.

03 forkFinally сообщает как об успехе, так и об исключении.

```
import Control.Concurrent
import Control.Exception (throwIO)

main = do
  done <- newEmptyMVar
  _ <- forkFinally (pure (42 :: Int))
         (putMVar done)
  result <- takeMVar done
  value <- either throwIO pure result
  print value
```

Ожидание takeMVar связано с завершением действия. Пауза threadDelay такой гарантии не даёт.

MVar и каналы

01 MVar содержит одно значение или пуст.

02 takeMVar ждёт значение, putMVar ждёт свободную ячейку.

03 Chan передаёт последовательность сообщений через очередь.

```
import Control.Concurrent
import Control.Concurrent.Chan

main = do
  ch <- newChan
  _ <- forkIO (writeChan ch "ready")
  message <- readChan ch
  putStrLn message
-- ready
```

Chan не ограничивает размер очереди. Медленный потребитель может привести к росту памяти.

STM: транзакции над общими данными

01 TVar хранит изменяемое значение внутри STM.

02 atomically фиксирует все изменения транзакции вместе.

03 При конфликте транзакция может выполняться повторно.

```
import Control.Concurrent.STM

transfer :: Int -> TVar Int
         -> TVar Int -> STM ()
transfer amount from to = do
  balance <- readTVar from
  check (balance >= amount)
  modifyTVar' from (subtract amount)
  modifyTVar' to (+ amount)
```

Предусловие примера: amount >= 0. Вызов: atomically (transfer 30 accountA accountB).

Счётчик с явным ожиданием потоков

01 Каждый поток атомарно увеличивает TVar.

02 Отдельный MVar сообщает о завершении каждого действия.

03 Сначала ожидаем все результаты, затем читаем счётчик.

```
counter <- newTVarIO (0 :: Int)
done <- replicateM 100 newEmptyMVar
forM_ done $ \signal ->
  forkFinally
    (atomically $ modifyTVar' counter (+1))
    (putMVar signal)
results <- mapM takeMVar done
mapM_ (either throwIO pure) results
readTVarIO counter >>= print
-- 100
```

Фрагмент из main. Полный файл: p07_stm_counter.hs. Импорты и обработка исключений находятся в src.

retry, orElse и границы STM

01 retry ожидает изменения прочитанных TVar.

02 orElse пробует альтернативу, если первая ветвь вызывает retry.

03 STM не гарантирует завершения и не исправляет ошибочную логику.

```
takeEither :: TQueue a -> TQueue a
           -> STM a
takeEither left right =
  readTQueue left
  `orElse` readTQueue right

-- Если обе очереди пусты,
-- транзакция ожидает изменения.
```

Если первая ветвь может завершиться, orElse выбирает её. Это не гарантия справедливого чередования.

Параллельные чистые вычисления

01 par создаёт возможность параллельного вычисления первого аргумента.

02 pseq задаёт порядок оценки до WHNF.

03 Слишком мелкие задачи могут увеличить накладные расходы.

```
import GHC.Conc (par, pseq)

pairSum :: Int -> Int -> Int
pairSum n m =
  let a = fib n
      b = fib m
  in a `par` (b `pseq` (a + b))

-- fib определена в полном примере
-- pairSum 20 21 = 17711
```

GHC.Conc относится к GHC и пакету base. Для высокоуровневых стратегий есть отдельный пакет parallel.

Сборка для нескольких ядер

01 -threaded подключает многопоточную среду выполнения.

02 -N2 разрешает два capability для исполнения Haskell-кода.

03 -s выводит статистику времени, выделения памяти и GC.

```
ghc -O2 -threaded -rtsopts `
  p09_parallel.hs -o demo

.\demo.exe +RTS -N1 -s -RTS
.\demo.exe +RTS -N2 -s -RTS

# Одинаковая нагрузка
# и одинаковый результат
```

Команды PowerShell. Обратная кавычка в конце первой строки продолжает команду на следующей.

Ленивость и отложенные вычисления

01 Вычисление начинается по мере востребованности результата.

02 Thunk представляет ещё не вычисленное выражение.

03 Разделяемое выражение обычно вычисляется один раз и обновляется.

```haskell
firstFive :: [Int]
firstFive = take 5 [1..]
-- [1,2,3,4,5]

answer :: Int
answer = const 7 (error "unused")
-- 7
```

Модель отложенных вычислений объясняет поведение. Оптимизации GHC могут устранять конкретные thunk.

Строгость: WHNF и полное вычисление

01 WHNF раскрывает внешнюю форму значения.

02 seq требует WHNF первого аргумента.

03 deepseq проходит структуру глубже согласно экземпляру NFData.

```
import Control.DeepSeq (deepseq)

-- Внешний конструктор списка известен
[undefined] `seq` 7
-- 7

-- Требуется также элемент списка
[undefined :: Int] `deepseq` 7
-- исключение
```

Строгое поле и foldl' также не означают автоматического полного вычисления всех вложенных данных.

Память и строгие накопители

01 foldl может накапливать цепочку отложенных операций.

02 foldl' вычисляет накопитель до WHNF на каждом шаге.

03 BangPatterns позволяет явно потребовать значение.

```haskell
{-# LANGUAGE BangPatterns #-}
import Data.List (foldl')

total :: [Int] -> Int
total = foldl' (+) 0

count :: [a] -> Int
count = go 0
  where
    go !n []     = n
    go !n (_:xs) = go (n+1) xs
```

BangPatterns включается отдельно. Для вложенного накопителя может потребоваться более глубокая строгость.

GHC, GHCi и абстрактная машина STG

01 GHC обычно компилирует программу в машинный код.

02 Core и STG являются промежуточными представлениями.

03 GHCi может исполнять байт-код и загружать объектный код.

```
-- Упрощённый путь native-сборки
Haskell
  Core
    STG
      C--
        машинный код

-- Код взаимодействует с RTS
```

STG описывает модель выполнения. Для обычной native-программы отдельная VM байт-кода не обязательна.

RTS и сборка мусора

01 RTS управляет потоками, кучей, стеками и исключениями.

02 GC освобождает объекты, недостижимые из корней.

03 Типичная конфигурация GHC использует поколения объектов.

```
-- Краткая диагностика
app +RTS -s -RTS

-- В отчёте:
-- bytes allocated
-- maximum residency
-- время MUT и GC

-- Лимит кучи:
app +RTS -M256m -RTS
```

maximum residency оценивает живые данные при крупных GC. Это не полная память процесса в ОС.

Удержание данных и диагностика

01 GC сохраняет объект, пока он достижим.

02 Ленивое замыкание или старая версия коллекции может удерживать большой объём.

03 Строгость и отказ от лишних ссылок решают разные причины расхода памяти.

```
-- План исследования
-- 1. Собрать одинаковую версию
-- 2. Запустить с +RTS -s
-- 3. Найти удерживаемые данные
-- 4. Изменить представление
-- 5. Повторить измерение

-- Больше выделено != больше живёт
```

Дескрипторы файлов закрывают withFile или bracket. На своевременную работу GC для этого не полагаются.

Итоги третьей части

01 IO связывает чистую обработку с консолью и файлами.

02 MVar, каналы и STM координируют конкурентные действия.

03 Ленивость, строгость и RTS определяют затраты выполнения.

```
-- Взаимодействие
IO a, withFile

-- Координация
forkFinally, MVar, atomically

-- Вычисление и память
par, foldl', deepseq, +RTS -s
```

Основы языка, организация кода и механизмы выполнения образуют три части курса.